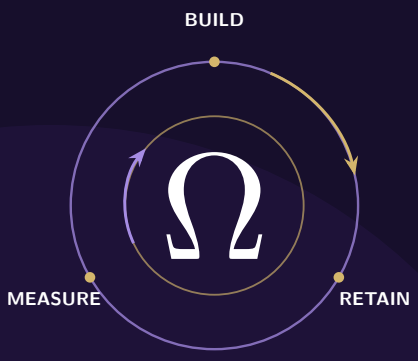
AGI ALPHA FLYWHEEL

Earn the Next Turn

From Reusable Work
to Accountable Succession



BUILD WHAT MAKES
THE NEXT MISSION BETTER.

SUCCESSOR Ω HARNESS

ACTIVATE. EVOLVE. SCALE.

Powered by TRANSFORMATIONAL INTELLIGENCE ∞

Vincent Boucher

Montréal, Québec, Canada / 24 October 2026

An additive experiment. Earlier editions and results remain intact.

Earn the Next Turn

From Reusable Work to Accountable Succession

Vincent Boucher · MONTREAL.AI

The result, before the ambition

Constructive succession executes; the registered net-benefit gate remains on HOLD. The system retains a checked response instrument, uses it on new loads, and invokes it to construct a distinct changed-network instrument. Eighteen such successors deliver 156 fresh, qualified parent-using outputs. Against native refactorization with conventional caching, however, the whole-participant CPU ratio is **0.9933**, with a paired-stream 95% interval of **[0.9669, 1.0265]**. This does not support the registered 5% benefit. All seven treatments qualify all 492 offered useful outcomes.

Abstract

Can completed work create a method that improves later work, helps construct a different capability, and pays for its own continuation? We introduce a reproducible causal succession experiment through the existing AGI ALPHA workbench. A native sparse solve supplies a reusable factorization as a byproduct of useful service. Fresh loads invoke that parent. Positive edge changes selected from the actual accepted predecessor then produce a low-rank response instrument through parent solves. Every output receives a separately implemented edge-balance and energy check; descendants inherit neither current proof status nor operational authority.

A source and protocol freeze precedes one confirmation campaign of 24 independent network streams, seven treatments and four workload/maintenance regimes. Controls retain ordinary caching, warm starts, competent native construction, an independently built equivalent parent, and equally enabled conventional reuse. All 3,444 offered useful outputs qualify. The flywheel constructs eighteen successors and qualifies 156 fresh outputs that execute their parent, improving on the prior edition's unactivated productive lineage. But the primary whole-participant CPU test fails: the point estimate is only 0.67% below native refactorization and its interval includes a loss. Conventional reuse satisfies the registered noninferiority guard, without evidence of unique agentic advantage. Actual released service quota funds 1,034.057 ms of successor dispatch; the quota originates in external endowment and does not prove self-financing growth.

The contribution is an executed, falsifiable boundary between useful inheritance, construction dependence, whole-system value and resource feedback. An offline browser companion, exact-source native implementation, protected confirmation record, unsuccessful development attempts, raw data and independent reanalysis are provided. Full research investment recovery, commercial growth and acceleration remain unestablished.

Keywords: executable inheritance; constructive succession; sparse numerical instruments; full-cost evaluation; resource custody; bounded autonomy; reproducibility.

What the evidence establishes

Question	Finding
Does the recurring loop run?	Yes. One launch executes 168 finite trials without manual phase triggers.
Does accepted work leave useful means?	Yes. Fresh solves use retained factors; actual B08 selects the successor's changed operator.
Does the parent construct a distinct child?	Yes. Parent calls produce the response basis; fresh changed-network outputs qualify.
Does net benefit survive the strongest comparison?	Not established. The registered 5% whole-participant CPU gate fails.
Does released capacity fund construction?	Yes, within a conserved externally supplied quota; incremental earnings are not proved.
Does the system accelerate or recover all investment?	Not established. No recursive discovery-rate experiment or complete monetary/human-cost evidence.

1 The question that survives a failed flywheel

A useful enterprise needs accepted work today and a defensible reason to invest in a better method tomorrow. The decisive object is an executable means of doing work: a response operator, a checked construction procedure, or a scoped verifier. A longer history, more tasks, or an ancestry pointer does not establish that this object improves the enterprise. This edition studies the entire transition from a useful answer to a reusable instrument, then from that instrument to a distinct successor, while preserving strong conventional alternatives.

The previous Strategy Succession edition is an essential negative control on the narrative [1]. Its frozen primary flywheel qualified 517 of 864 offered outcomes, tied its equally funded conventional learner, and fell below the fixed portfolio's 591. No primary capability was admitted. No fresh repair both invoked its nominated parent and qualified. Diagnostic inheritance reduced work relative to exact reconstruction, but the cheaper parent-free alternative qualified more work. Those results remain unchanged; this paper does not reinterpret them as growth.

The new contribution is a *causal succession test with resource custody*. It has four separable obligations:

1. Deliver an independently checked result through the supported Alpha workbench.
2. Retain an input-specific executable instrument and demonstrate actual use on fresh work.
3. Construct a different operator using that instrument and the actual accepted predecessor state, then validate the descendant on fresh requirements.
4. Account for whether the entire sequence earns its cost against competent direct construction and reuse, and trace any resource reassignment without calling external endowment new wealth.

This design intentionally gives conventional methods every legitimate opportunity. The equally enabled reuse arm executes the same available policy; the native-refactor arm retains factors and builds a fresh factor for a changed operator. The parent-rebuilt and equivalent arms isolate construction dependence. None is an institutional proof of intelligence. The experiment can establish implemented reuse, a causal construction path, scoped numerical correctness or a measured resource advantage. These are different scientific claims.

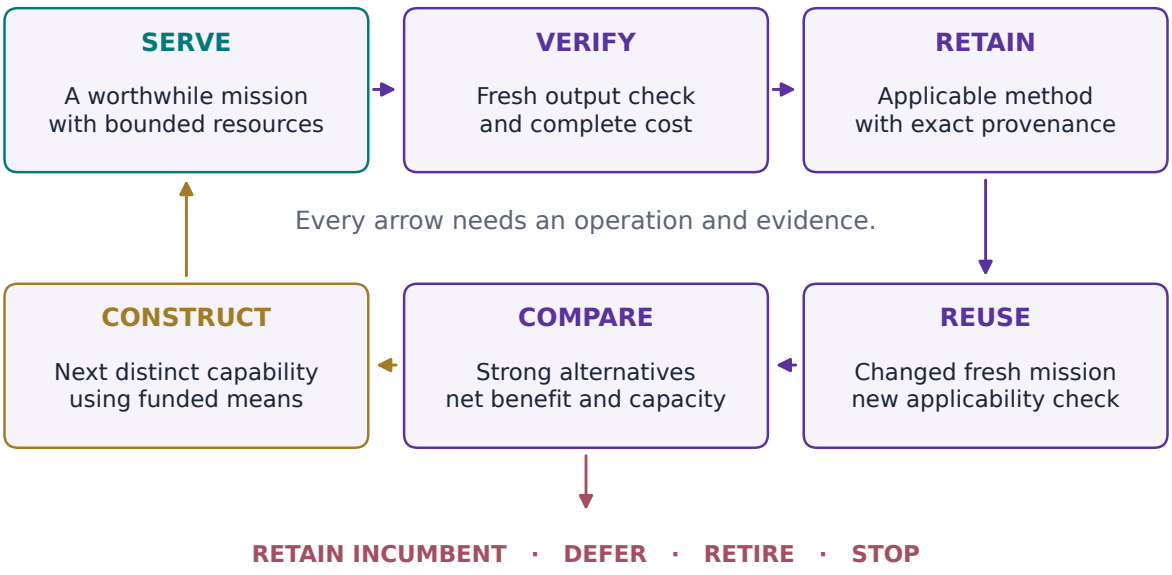


Figure 1: The obligation at each turn. A checked useful result supplies evidence and a candidate method. Reuse and successor construction remain subject to independent output checks and separate cost decisions.

1.1 Reconcile the failure before changing the experiment

A read-only audit rechecks 96 primary checkpoints, 48 diagnostic checkpoints, twelve diagnostic raw archives and all 52 files in the earlier freeze. At each primary acquisition stage, a 500,000-unit ceiling protected

401,024 units for qualification: 80.20% of the budget. Search had at most 98,976 units before other control reserves. The eight-method successive-halving race required up to seventeen paid calls to finish six training cases, but every primary stream stopped after eleven to thirteen calls. All 140 paid prefixes were valid and untruncated. Twelve subsequent calls were refused before computation. Qualification and admission were never reached.

Of 96 menu entries, 58 were ordinary eliminations, 26 were incomplete valid prefixes, and twelve were unfunded calls. Reserved qualification allowance was not spent work. Raising the overall stream budget to two million units left this acquisition ceiling unchanged. Marking a stage completed before acquisition also made that failure terminal; later stages deferred without an accepted parent.

The prior cost model additionally charged 5,238,180 units to checkpoint rewriting, 44.7% of primary flywheel work. Research and service shared a cumulative account. These observations support an allocation mismatch, expensive persistence and weak isolation of promised delivery. They do not demonstrate a corrupt numerical result or prove that no transferable scheduling method exists. The diagnostic nevertheless warns against treating allocation repair as sufficient: inherited work averaged 888,009.50 units for 36 qualified outputs across 48 requests, while the parent-removed conventional alternative averaged 225,339.33 and qualified 37.

The earlier package also recovered SUCCESSOR Ω—Strategy Laboratory v0.1.0, archive SHA-256 beginning `c0df2ac39ddcb32d`, and replayed its unchanged frozen policies under CPython 3.13.5. All six attempts and 27,648 cases reproduced their non-timing outputs and judgments; a separate artifact audit checked 193,536 policy/control artifacts. This verified historical replay did not rerun acquisition search, establish equal total computation against specialists, or qualify the present flywheel. The full historical evidence remains in the prior edition; byte-bound status and replay records are copied into this edition's audit docket.

The new study therefore changes the domain, construction representation, accounting, causal predecessor, persistence and comparators. It uses new identities and new tasks. The earlier evidence is an audited motivation, not a reused confirmation set.

1.2 An operational interpretation of the Out stages

Stage	Inspectable operation and evidence
Identify	Bounded three-domain screen; fixed mission stream and explicit scope.
Out-Learn	Accepted response, edge checker, development cost table and retained regression counterexamples.
Out-Think	Compare low-rank construction with native refactorization using a frozen forecast.
Out-Design	Construct factor or update arrays with a typed interface, operator identity and dependencies.
Out-Strategise	Reserve service, research or released quota; defer when allowance or scope is insufficient.
Out-Execute	Original workbench dispatch, native computation, independent numerical check and exact receipt.
Out-Evolve	Retain scoped instruments, reject stale reuse, preserve the incumbent and select the next permitted request.

Table 1: An implementation role for the conceptual stages. These names do not imply seven independent models or an unconstrained discovery process. The frozen methods and data are the operative mechanism.

2 Use the foundations as engineering obligations

The foundational AGI ALPHA paper supplies an organizational substrate: bounded work, explicit evaluation, reusable means and attributable evidence [2]. TRANSFORMATIONAL INTELLIGENCE ∞ contributes a sharper experimental demand: change the means of satisfying a purpose while leaving the external acceptance contract fixed, then remove the proposed change to test its contribution [3]. Here the purpose remains a checked network response. The executable method can change from solving a fresh linear system to invoking a retained factorization, then to applying a scoped update instrument. A conventional implementation receives the same options. The symbol ∞ names the research programme; it grants neither unbounded resources nor a presumed result.

Foundation	Implemented obligation and boundary
Possibility Foundry [4]	Price the enabling instrument and its builder; test the inherited instrument in a distinct construction. Include the competent native builder. Its own instrument results do not qualify this experiment.
Mission-Sovereign Successors, v5.0.0 [5]	Compare retain, construct, use the available specialist, defer and stop. Bound the smallest decision-changing trial. No purchase, new service contract or institutional designation is implied.
Neural-Symbolic SUCCESSOR Ω, requested v0-URL document [6]	Bind an executable interface to its purpose, dependencies, predictions, falsifiers and provenance. Keep WORLD, POLICY, PROOF and AUTHORITY distinct. A numerical adapter does not instantiate the proposed neural world model.
Gym, Specialist ASI & Successor Ω, v2.1.0 [7]	Freeze candidate, comparator and checker before fresh confirmation; compare complete workflows and keep admission scoped. A correct gate can reject every candidate.
SUCCESSOR Ω× AGI JOBS, v2.0.0 [8]	Bind work orders to input, purpose, source, budget, expiry and dependency evidence. Invalidate stale descendants. Local drafts supply no execution or settlement authority.

Table 2: Foundational ambitions become finite obligations. The cited documents' experiments remain source-reported unless separately replayed; their conclusions are not inherited by this study.

Exact edition. The neural-symbolic document inspected here is the requested `neuralsymbolicsuccessorome gav0.pdf`, dated 25 August 2026 and identifying itself as a *Definitive Corporate White Paper*. Its 59 pages and SHA-256 beginning `b1c0d9875bdbdb24` are bound in the foundation manifest. Prior Strategy Succession inspected the different v1-URL document. Equal page counts are not equal artifact identities; this edition makes no silent substitution.

2.1 Every Out-stage has an observable job

Identify chooses a declared worthwhile request or a repeated bottleneck and checks its scope and remaining accounts. **Out-Learn** measures the construction, checking and reuse costs on bounded development cases. **Out-Think** challenges a reuse claim with a fresh load, changed operator or removed parent. These stages preserve failures and counterexamples rather than revising the final evaluator [9, 10].

Out-Design constructs an executable factor or update package with a current dependency record. **Out-Strategise** compares the available specialist, conventional reuse, prospective adoption cost and remaining recurrence before allocating more work [11, 12]. **Out-Execute** dispatches the fixed adapter through the existing Alpha workbench and obtains a new balance/energy check. **Out-Evolve** retains, declines or invalidates the asset and tests whether it contributes to a distinct construction [13, 14].

The source papers' physical calibration, finite-code theorems, bit-parallel machine model, institutional proof network and commercial programme remain outside this experiment. Their specific role here is to demand purpose preservation, useful construction, full accounting and separately checked succession. Familiar numerical methods remain familiar numerical methods; assigning them an Out-stage does not create a novel algorithm.

3 A response instrument that can build a successor

3.1 Choose transfer before choosing confirmation

The prospectively bounded development charter compares three mission families: resource scheduling, repeated shortest paths, and grounded sparse-network analysis. It fixes four screen seeds, two sizes per family, twelve repeat queries and one candidate mechanism per family, with at most 24 screen configurations. The decision criteria precede the outcomes: a useful first result, genuinely fresh inputs, independently checkable outputs, a distinct executable successor, a competent cheap builder, and a plausible unfavorable shift. The selected domain permits at most six algorithm variants and three complete development campaigns. Old confirmation cases remain historical results, not a newly declared development set.

Sparse-network analysis has a direct transfer hypothesis. The expensive common object is an operator; the changing request is its applied load. A response instrument can avoid repeated factorization. A later change in a small number of edges requires a different operator and can create a distinct instrument using the first instrument's solve interface. These are standard numerical opportunities. They provide an honest test of whether discovery, qualification, integration, persistence and upkeep erase the apparent benefit.

3.2 Useful work and a separate numerical checker

For n nodes, let e_i be the ith coordinate vector. An edge $a = (u, v)$ has incidence vector $b_a = e_u - e_v$ and positive conductance w_a. With positive diagonal grounding G, define

$$A = G + \sum_{a \in E} w_a b_a b_a^{\mathsf{T}}, \qquad Ax = f. \tag{1}$$

The output x is the response to a new load f. In the synthetic computational model it can be interpreted as node potentials or a grounded steady-state response; no physical network was built or calibrated. The useful result includes node values, edge flows, current input identity and numerical check evidence.

The checker is implemented separately from matrix assembly and the solver. It accumulates each node's ground term and incident edge flows directly from the supplied edge list, using compensated summation. It also compares dissipated energy with applied work:

$$r_i = g_i x_i + \sum_{(i,j) \in E} w_{ij}(x_i - x_j) - f_i, \tag{2}$$

$$\mathcal{E}(x) = \sum_i g_i x_i^2 + \sum_{(u,v) \in E} w_{uv}(x_u - x_v)^2. \tag{3}$$

Qualification requires both a normalized maximum residual and normalized energy discrepancy no larger than 10^{-9}. The schema rejects nonfinite data, nonpositive conductances, invalid endpoints and grounding below 0.1. The accepted domain has at most 65,536 nodes and 1,000,000 supplied edges; the experiment uses the smaller frozen sizes specified in its protocol.

Positive grounding and conductance make A symmetric positive definite. Hence a residual is relevant to the fixed numerical problem: for the exact solution $x^\star$, $A(x - x^\star) = r$, and $\|x - x^\star\|_2 \le \|r\|_2 / g_{\min}$. This algebraic bound does not validate a physical interpretation, source annotations or real demand. The second energy check is a useful consistency check, not an independent real-world oracle. The separate implementation reduces shared-code error while retaining the limitations of one commissioned team, one host and a common mathematical specification.

3.3 Start with an available specialist

The initial service uses the justified native COLAMD sparse factorization and retains its already-computed factor as a byproduct of the accepted result. There is no second artificial builder charge for the same factor. The comparison set includes native sparse direct factorization and preconditioned conjugate gradient; ordinary warm starts and conventional factor reuse are permitted. SciPy's `splu` invokes SuperLU and returns a reusable solve object; its documented column orderings include COLAMD and minimum-degree variants [15]. The conjugate-gradient path uses a diagonal preconditioner, the previous accepted solution as an allowed initial guess, a fixed tolerance and a finite iteration limit [16]. The experiment does not handicap the direct method by erasing legitimate previous work. The frozen comparison distinguishes standalone specialist performance from total workbench and custody cost.

The software relies on established SciPy algorithms [17]. No new linear solver, factorization, low-rank identity or uniquely agentic advantage is claimed. The contribution lies in the causal mission sequence, executable inheritance, protected resource accounts, complete comparisons and explicit decision boundaries.

3.4 Mission A and the retained parent

Mission A delivers a checked solution on A and the native factor used to produce it. Candidate parent capability F stores an exact operator identity, dependency versions, permutations and sparse factors, exposing

$$F: \; f \longmapsto x \approx A^{-1} f. \tag{4}$$

It is a method for responding to new loads, not a stored answer to A. Its native construction cost is already charged to A; recording it again as separate research would double count. Retention in the research catalogue requires a checked output and an explicit scope; scientific validity, prospective product adoption and recovery of investment remain different decisions.

The asset interface binds dimensions, operator hash, chosen ordering and dependency versions. Its life-cycle cost includes construction, current-output checking, integration, retrieval, storage and maintenance. Portable arrays are stored without executable pickle objects. Reload verifies the byte receipt, metadata, array shapes, permutations and triangular structure before using sparse triangular solves [18]. The native live factor and portable restored factor may have different execution costs; their mathematical interface alone does not establish equal operational performance.

3.5 Mission B: fresh work, actual use and a real predecessor

Each B request supplies a new right-hand side on the same accepted operator. Before reuse, the adapter rechecks operator identity and scope, then records actual parent calls. Every returned response receives the current edge-list checker. The distinct successor requirement is selected from the actual accepted preceding B output: edges are ordered by absolute realized flow, with a fixed index tie-break. The chosen positive conductance increments and the preceding input, solution and receipt digests enter the successor identity. A separately generated baseline cannot stand in for this accepted state.

This state-dependent rule is itself frozen. It makes the input to construction depend on useful completed work without allowing a failed candidate to redefine the acceptance test. Across treatments, the design retains actual prior-state provenance and reports any resulting requirement differences; apparently identical ancestry labels do not establish equivalent tasks.

3.6 Mission C: construct a distinct update instrument

For k changed edges with positive increments δ_j, form

$$U = [\sqrt{\delta_1} b_{a_1}, \dots, \sqrt{\delta_k} b_{a_k}], \qquad A' = A + UU^{\mathsf{T}}. \tag{5}$$

The constructor invokes the parent on the columns of U, obtaining $W = F(U)$. It then retains $S = I + U^{\mathsf{T}}W$ and exposes the distinct operator

$$H(f) = F(f) - WS^{-1}U^{\mathsf{T}}F(f) \approx (A')^{-1}f. \tag{6}$$

This is the established Sherman–Morrison–Woodbury construction [19]. The original parent cannot correctly answer an arbitrary load on A' by itself. Its execution helps construct H, and each fresh use of H again calls the parent. The original operator, exact increments, actual accepted B receipt, parent identity and dependencies are bound to the successor. Construction-call and delivery-call counts are recorded separately.

The implementation retains an incumbent or uses native refactorization when the registered scope policy declines the update. A grounding change invalidates both earlier operator scopes. Nonpositive edge conductance is refused as unsupported. Such cases are preserved as outcomes; they are not silently replaced by easier confirmation inputs. A failed construction must leave a competent service alternative available.

3.7 Causal comparisons and three different improvements

Execution improvement asks whether fresh accepted work costs less or achieves more at the same resource limit. **Construction improvement** asks whether retaining F makes building or qualifying H better than reconstructing the parent, independently constructing an equivalent asset, or using a strong parent-free solver. **Discovery improvement** would require a separately evaluated improvement to the search for such methods. It is not inferred from the first two.

The inherited, reconstructed, equivalent, native-refactor and equally enabled conventional treatments retain the same input rights and reasonable optimization opportunities. Conventional reuse may invoke the identical method and erase any named-controller advantage. Parent removal allows reconstruction; the parent-free specialist remains entitled to solve A' directly. A parent-use count establishes that an interface was invoked. Its causal value requires the corresponding outcome/cost contrast. A cheaper exact reconstruction comparison alone cannot establish practical adoption against a stronger direct alternative.

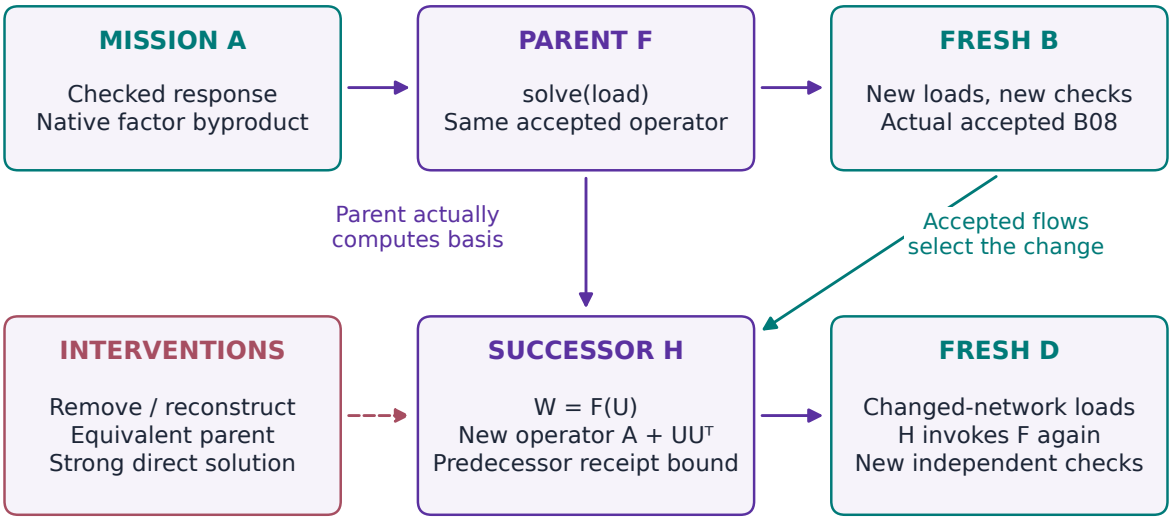


Figure 2: An operational lineage. The accepted B response determines the changed requirement; the retained parent actually computes the response basis used to construct the new instrument. Fresh D requests use the changed operator and receive new checks. Counterfactual treatments intervene on the executable parent, not merely its ancestry label.

3.8 Separate validity, adoption and investment

For asset a, let C_{build} be construction and initial qualification cost, m the prospective number of applicable uses, c_a current cost per qualified use including upkeep, and $c_\star$ the best available alternative. A simple break-even condition is

$$m(c_\star - c_a) > C_{\text{build}}, \qquad c_\star > c_a. \tag{7}$$

The actual decision uses the registered uncertainty and quality conditions; this expression explains why recurrence matters. Once construction is sunk, marginal adoption instead compares future burdens. The provisional research-use rule compares frozen development medians for the construction kernel plus the declared remaining solves: $1.20\,\widehat{C}_{\text{candidate}} < \widehat{C}_{\text{native}}$. This applies a 20% surcharge to the candidate forecast, equivalent to requiring more than a 16.7% reduction relative to the native forecast. It does not predict complete persistence, checking, transport or maintenance cost, and therefore is not product admission. The protected confirmation applies the stronger whole-workflow product rule: every required output must qualify, the paired stream interval must support at least a 5% CPU benefit over native refactorization, and the workflow must satisfy the registered 10% noninferiority margin against equally enabled conventional reuse. Otherwise product status remains HOLD. Passing a cheaper kernel comparison cannot substitute for this complete-system gate. Investment recovery must additionally charge exploration, rejected variants, complete checking, integration, maintenance, recovery and attributable human effort. Unknown human durations and monetary cost remain unknown. A candidate can be numerically valid and worth adopting prospectively while its full research investment remains unrecovered. Conversely, valid inheritance can be an expensive liability.

4 A frozen experiment with strong alternatives

4.1 Development, freeze and protected confirmation

The development charter fixes three domains, four screen seeds, two sizes per domain and twelve repeat queries. The screening results are preserved, including a very cheap scheduling baseline and effective ordinary shortest-path reuse. Sparse-network analysis is selected for its inspectable construction mechanism: an accepted response can select a different operator and its parent solve can construct that operator's response

instrument. The screen is small and purposeful, not evidence that this domain is commercially optimal or uniquely suitable.

Six numerical variants are investigated on four declared development seeds: two native sparse orderings, cold and warm iterative solves, and two low-rank sizes. Development calibrates a fixed cost forecast. It also exposes the substantial checking, hashing, dispatch and persistence burden omitted by an isolated kernel timing. Three full campaign attempts and a bounded remainder are retained. The third campaign stops on a restore taking 154.044 ms against a 150 ms reservation. Its remainder stops on 274.58 ms against 250 ms. Final conservative resource limits are set before confirmation; those unsuccessful attempts are not overwritten or counted as final passes.

Independent review adds consequential counterexamples. Integer grounding values initially caused diagonal assembly truncation; malformed updates and incoherent restored identities needed refusal; an existing-output failure handler could overwrite prior evidence; mutable factor filenames could overwrite the incumbent. The corrections, pre-fix examples, narrow regression checks and reviewer authorship overlap are preserved. No fourth efficacy campaign is run to seek a preferred outcome.

The 32-file freeze is created at 01:09:39 UTC on 25 October 2026, corresponding to the evening of 24 October in Montréal. The frozen protocol archive is saved before a prospective notice in issue #4850 [20]. Fresh network seeds are generated only afterward. The single campaign uses this closure unchanged, including both aggregation implementations, the checker, calibration, dependencies and browser execution modules. The browser's small arithmetic demonstration is a distinct experiment.

4.2 Tasks, regimes and treatments

Twenty-four independent network streams are assigned evenly to four fixed regimes. Conductances are sampled uniformly from $[0.5, 2]$ on the declared grid, grounding is 0.1, and successive loads are fresh normal draws. Each stream delivers A, eight B requests, C, fresh D requests and a changed-grounding request. A negative conductance request is a separate prospectively unsupported case. It must be explicitly refused.

Regime	Nodes	Rank	Fresh D	Useful	Changed requirement
Recurring	2,304	2	12	23	Same operator, fresh loads, then selected edges
Rare	2,304	2	2	13	Lower recurrence over which to recover costs
Wide change	2,304	32	12	23	Broader construction; native alternative remains available
Maintenance	4,096	2	12	23	Larger workload and matched memory eviction at D06

Table 3: Six independent streams per regime. Correlated loads within one stream are not treated as independent trials. There are 492 offered useful outcomes per treatment.

Treatment	Information, construction and reuse rights
Flywheel	Retain A's factor; use frozen forecast for provisional low-rank construction; spend released quota.
Conventional reuse	Same information, builders, checks, policy and resources; no AGI label confers an advantage.
Direct specialist	Warm-start, diagonally preconditioned CG; retain matrix representation and previous accepted solution. This isolates absence of specialized factors.
Parent rebuilt	Discard the retained parent at C, reconstruct it with the competent native builder, then build the same kind of child. A construction ablation.
Equivalent parent	Independently construct a minimum-degree-ordered equivalent factor at C; include its actual cost. A construction ablation.
Native refactor	Retain A's factor, construct a fresh native factor for C and cache it for all later loads. The primary strong comparator.
No reinvestment	Retain useful factors; no transfer to successor research. The native changed-operator service remains available.

Table 4: Every treatment uses the workbench and current checker. Reconstruction ablations are interpreted alongside the competent parent-free construction, so exact rebuilding cannot become an artificially weak headline comparator.

C's edge choice uses the actual accepted B08 response in each arm. All resulting task identities are compared across arms. A divergence would be a reported comparability defect. The fixed offered denominator includes unattempted missions after failure. A failed trial writes its partial inputs, solutions, receipts and summary; it cannot disappear because its loop ended early. Each trial directory is created atomically and cannot be silently reused to overwrite accepted history.

4.3 Three admissions and one primary endpoint

Correctness concerns the exact output and operator scope. **Provisional research use** additionally requires a development kernel forecast: candidate construction plus remaining use, with a 20% surcharge, must beat native construction plus use. This forecast does not estimate every lifecycle cost; it is explicitly not product admission. The reconstruction ablations may execute outside that forecast to identify construction dependence.

Product admission requires fresh whole-participant evidence. Define, for paired stream costs T_i,

$$R = \frac{\frac{1}{24}\sum_i T_i^{\text{flywheel}}}{\frac{1}{24}\sum_i T_i^{\text{native refactor}}}. \tag{8}$$

The primary gate requires the upper bound of a two-sided 95% confidence interval for R below 0.95, all offered flywheel outcomes qualified, and no trial/resource violation. A conventional-reuse guard requires its corresponding upper CPU-ratio bound below 1.10, with full matched comparator qualification. Failure leaves product status on HOLD. Even a pass would not recover unknown human and monetary research investment.

The registered analysis resamples complete paired network streams within their four strata, six draws per stratum, for 10,000 bootstrap replicates using seed 20261025. Arm order is shuffled with seed 20261024. Other contrasts, regime-specific results and construction timings are descriptive. There is one confirmation campaign, no optional exclusion and no post-outcome retuning or rerun. Protected tasks and authority rules are outside the candidate's closed data-construction interface; this is an integrity-bound same-host design, not a separately administered evaluation enclave.

4.4 Cost and resource boundaries

The primary cost is measured Python process CPU plus actual reaped workbench-child CPU. It includes stream setup, matrix assembly, hashing, checking, dispatch, retention, failed work, restoration and writing raw artifacts. Wall time is measured separately. Kernel CPU, checker CPU and action CPU are nested components, not quantities to add again to the total. Final report serialization and package production are research overhead outside the participant measurement. Import, development, review and independent audit costs remain separately attributable. Unknown human duration, model fees and monetary cost are represented as unknown.

Per stream, the final dispatch accounts contain 10,000 ms of service allowance and 1,000 ms of externally supplied improvement allowance. Released quota starts at zero and is capped at 500 ms. Each accepted B may transfer up to 50 ms of its unused 1,000 ms service reservation; eight B requests can therefore transfer 400 ms. C reserves at most 400 ms from that account. Restore reserves 600 ms from improvement. Shared startup, custody settlement and aggregate artifact writing have a separately measured 2,000 ms external control bound. Every treatment receives the same opportunity except the explicit no-reinvestment ablation.

An observed native-call overrun is charged and refused; these accounting controls do not physically preempt an indivisible BLAS call. Most critically, unused quota is externally supplied capacity. Its transfer and debit show operational resource feedback. Only the paired timing comparison can establish an incremental saving, and neither observation creates cash or proves a self-financing enterprise.

5 Confirmation: useful succession, held product admission

The single frozen campaign ran from 01:10:38 to 01:15:28 UTC on 25 October 2026, taking 289.289 s wall time. Measured campaign CPU was 171.115 s in Python and 123.638 s in its workbench children. All 168 registered trials completed without a source change, manual phase trigger, trial failure or declared resource violation. The post-run freeze check reverified all 32 files. No additional confirmatory campaign was started.

5.1 Complete service coverage, without an expanded envelope

Every treatment qualified 492 of 492 offered useful outputs. Across the campaign, 3,444 useful outputs and 42 maintenance outputs passed the independent numerical checker. All 168 unsupported negative-conductance requests were explicitly refused. Because the competent controls achieved the same coverage, this result shows reliable implementation in the declared envelope; it does not show a newly feasible mission set.

Treatment	Qualified	Mean CPU (s)	Successors	Fresh child uses
Flywheel	492/492	1.731	18	156
Conventional reuse	492/492	1.745	18	156
Direct warm CG	492/492	1.764	0	0
Parent rebuilt	492/492	1.752	24	228
Independent equivalent	492/492	1.759	24	228
Native refactor + cache	492/492	1.742	0	0
No reinvestment	492/492	1.759	0	0

Table 5: Whole-participant CPU per independent stream. "Successors" denotes low-rank instruments; the native refactor arms create ordinary current-operator factors. The two forced construction ablations are diagnostic, so their 24 children are not 24 economic promotions. All treatments can retain legitimate ordinary state.

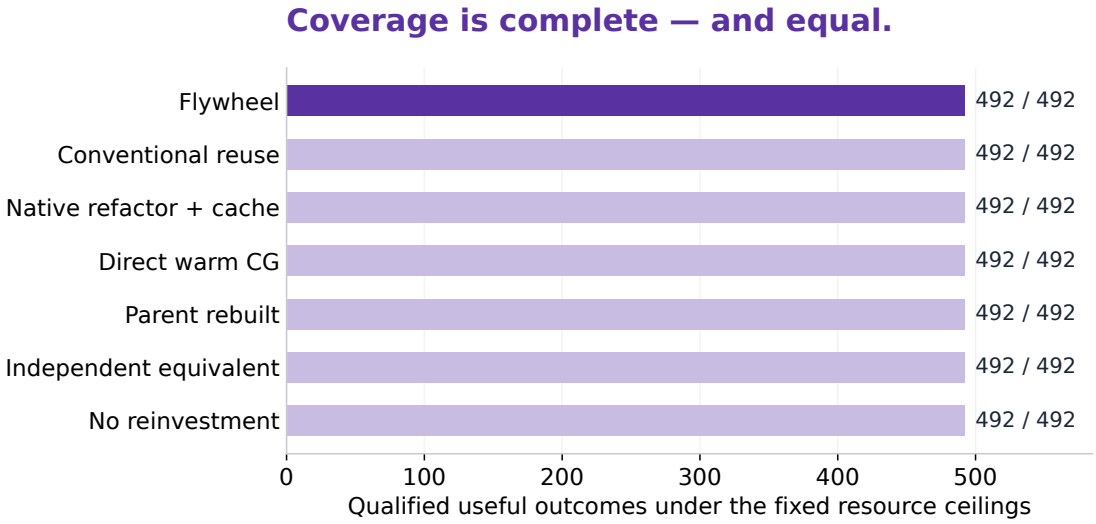


Figure 3: The same qualified coverage under the fixed resource ceilings. Useful computation is real, but more inherited assets do not expand the tested mission envelope beyond competent conventional execution.

5.2 The primary gate fails

The flywheel's mean whole-participant CPU is 1,730.732 ms, compared with 1,742.402 ms for native refactorization and caching. The ratio is $R = 0.9933025$, with registered paired, stratified bootstrap interval $[0.9668841, 1.0264574]$. The point estimate is 0.67% lower; the interval spans both benefit and loss and does not support the required 5% improvement. **Product admission is HOLD.**

Against equally enabled conventional reuse, the ratio is 0.9915906 with interval $[0.9639719, 1.0191137]$. This passes the prespecified 10% noninferiority guard. The conventional policy has the same capability choices; its agreement with the flywheel is expected evidence of enabled conventional reuse, not evidence of unique AGI superiority. The direct-specialist contrast is also inconclusive end to end: ratio 0.9812102, descriptive interval $[0.9437037, 1.0200646]$.

5.3 A real successor: what transfers and what it costs

In the eighteen eligible low-rank flywheel streams, C makes one batched parent call to compute the two-column response basis, followed by a second parent call while qualifying the changed-network response.

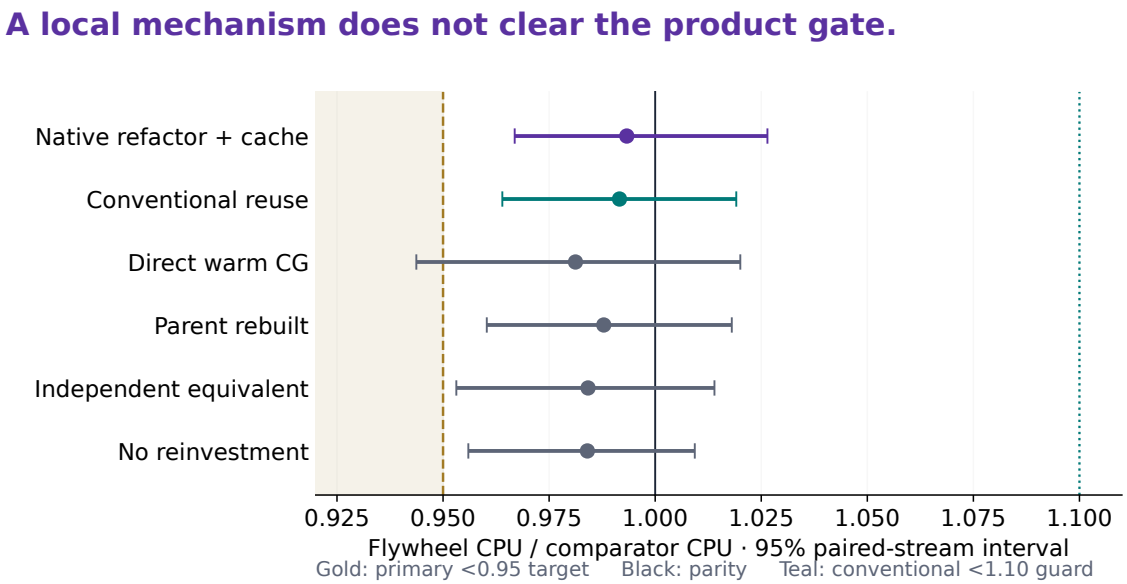


Figure 4: Ratios of mean whole-participant CPU, with paired-stream percentile intervals. The primary native-refactor interval does not clear the registered 0.95 threshold. Other contrasts are descriptive; conventional reuse also has a separately registered 1.10 noninferiority guard. Lower ratios favor the flywheel.

Later fresh D requests make 156 additional parent calls and all qualify. The independent audit ties the selected edges to actual accepted B08 flows and finds no paired input divergence across treatments. This closes the earlier edition's gap between a stored parent pointer and a fresh qualified output that actually uses its parent.

The first registered stream, seed 684407853, is a concrete example rather than a selected best case. A solves 2,304 nodes and 4,512 edges, with relative residual 2.44×10^{-15}. The retained parent identity begins `88d25bbb31dd`. B08 has a new load and a different checked response. Its two highest-flow edges, indices 2991 and 3320, receive the prescribed positive increments. C constructs child `19ec0f844656`, binding B08 receipt `f492b0100a47`; its persisted update arrays occupy 77,408 bytes. Fresh D01 uses both that child and its parent, with relative residual 1.52×10^{-15}. The subsequent grounding change refuses old scope and uses the specialist; negative conductance is refused without a result.

A clean construction intervention compares the same low-rank constructor with and without a retained factor. In the recurring regime, C's measured algorithm phase averages 1.61 ms with inheritance and 7.88 ms with native reconstruction; the equivalent-parent version averages 6.43 ms. The phase includes construction, one qualification solve and persistence. It is not a pure factorization benchmark. The end-to-end C action averages 51.57, 54.27 and 58.12 ms respectively, showing how much overhead can dilute that local difference.

Native C averages 10.21 ms in its algorithm phase and 66.53 ms for its action in that regime. However, its code path performs an additional full input-scope scan that the parent-based constructor replaces with bounded parent/update checks; both receive the same final checker. Therefore this secondary gap mixes construction and validation work. It must not be advertised as an isolated algebraic speedup against an equally optimized native constructor. The primary whole-workflow result already remains held.

5.4 Maintenance and the simpler alternative

The flywheel retains the native alternative in all six wide-change streams rather than building the proposed low-rank instrument. In the maintenance regime, its mean full cost is 2,569.456 ms, above native refactorization's 2,538.358 ms and direct CG's 2,470.167 ms. Portable factor validation and restoration are substantive work: flywheel restore actions average 152.02 ms, while direct CG's matched recovery action averages 81.29 ms. In the rare regime, flywheel CPU is also slightly above native refactorization, 1,088.842 versus 1,086.199 ms. These are descriptive regime findings with six streams each, not independently significant subgroup claims.

Only 3.274 s of the flywheel's 41.538 s total participant CPU lies in the recorded algorithm phases; these phases themselves include preflight and persistence work. The independent checker consumes 4.860 s, work-

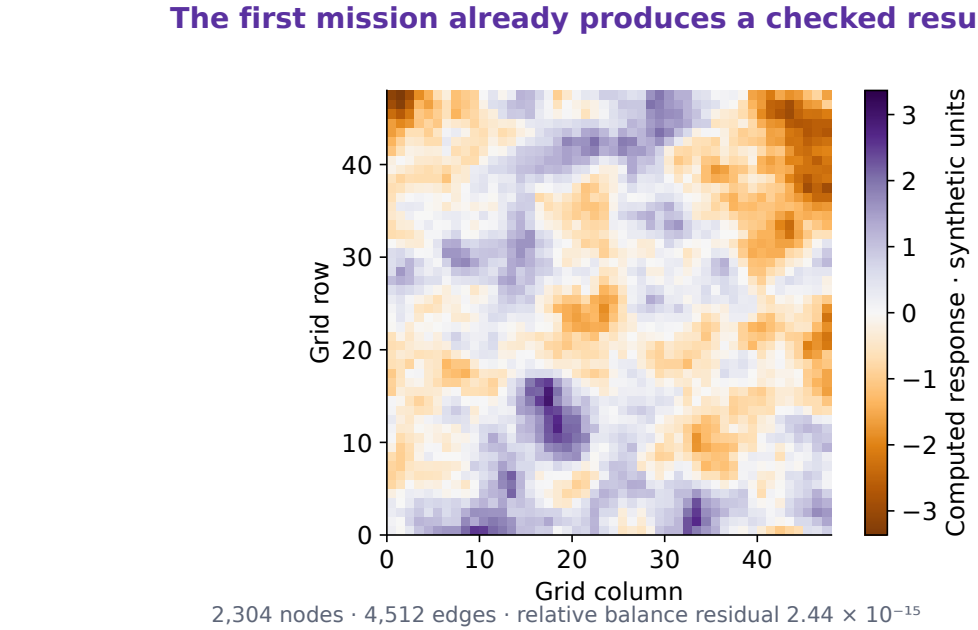


Figure 5: A useful result before any succession claim: the first registered stream's independently checked node response. The full inputs and output vector are preserved. Colors represent this synthetic numerical solution, not an observed physical network.

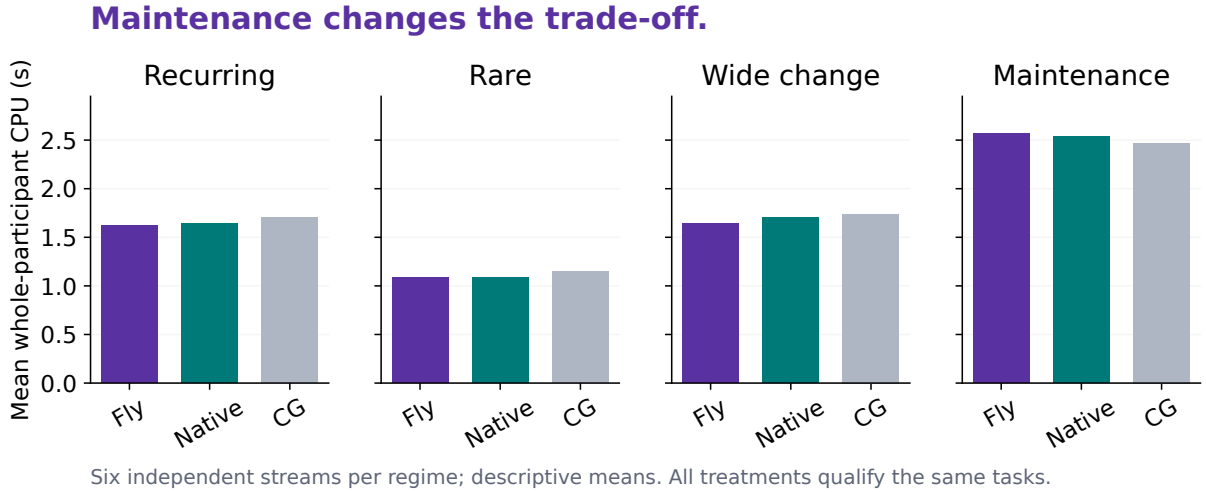


Figure 6: When reuse helps locally and when its complete burden remains uncompetitive. Every regime has equal qualified coverage. Maintenance can make a simpler solver preferable even when the inherited instrument is correct.

bench child CPU 15.579 s, and shared setup/custody/aggregate-artifact work 7.129 s. Other Python binding, serialization and transport work fills the remaining difference. These are disjoint components only after subtracting the nested measurements correctly; action CPU must not be added again to its constituent costs. A faster solve does not automatically make the complete service proportionally faster.

5.5 Resource feedback is executed, not self-financing growth

Across 24 flywheel streams, accepted B work transfers 9,600 ms of unused service quota to the released-capacity account. The eighteen successor actions debit 1,034.057 ms of actual dispatched CPU. Construction is therefore funded within the reassigned allowance and does not borrow the separate research account. The native and direct controls receive the same release opportunity; an abundance of unused reservation is not an inherited advantage.

These transfers originate in externally supplied capacity. The paired comparison does not establish the required incremental productivity gain, and a 1,034.057 ms child debit cannot be relabelled profit or recovered research investment. The remaining released allowance is unused endowment. No monetary margin, customer demand, paid adoption or increasing improvement rate was measured.

5.6 Independent verification and its limits

The frozen second implementation audits 168 trials, 3,654 raw records, 3,486 accepted numerical outputs and 5,334 artifact identities with zero reported issues and zero paired-input divergences. It reproduces the exact primary point ratio. Its independently arranged bootstrap draws give interval $[0.9672105, 1.0271364]$; the small endpoint difference is Monte Carlo/resampling-layout variation, not another experiment. Both frozen analyses leave product admission on HOLD. Its conventional interval $[0.9650167, 1.0194604]$ also passes the registered guard.

The checker and aggregator are separately implemented, but their reviewer belongs to the same commissioned team and helped correct executor validation before the freeze. CPU observations cannot be reconstructed from hardware after the fact; the audit reconciles their stored accounting boundaries. Parent calls are producer instrumentation, independently attributed to output and lineage records, not an external instruction trace. These limits are part of the evidence, not grounds for upgrading institutional authority.

6 One finite start, accountable operation

6.1 The actual Alpha execution path

Each mission uses the released Alpha v1.42.0 `createWorkbench` extension from source commit `c352bad0a0941fdccd490a7224e3efd1b8de45d5` [21]. The eight-file execution closure is unchanged; the current-main entry file is byte-identical to that release. A ninth vendored file is the existing transactional browser history store, obtained at current-main commit `5499495911bf8822ac0aa8c9007f00789c1132b5`. Its identity is recorded separately from the execution closure. A persistent local Node bridge validates and freezes a request, invokes its registered local adapter, requests the fixed Python callback, binds the returned artifact and inspects the original workbench pack. Numerical computation therefore follows workbench dispatch. It is not computed first and retrospectively wrapped as an agent result.

The concrete seam is `createWorkbench(adapters).run(spec,options)`, followed by `inspectPack(pack)`. The Python client supplies mission identity, purpose, exact inputs, the fixed callback and the enclosing source hash. The callback returns a finite report with validity, qualification and measured output; it cannot confer the independent institutional proof that the upstream template lacks.

The finite research recipe supplies the declared mission sequence. It is not a newly installed scheduler. The maintained native `CampaignOperation` uses the existing source-clause study queue and fixed selectors; it does not offer this numerical family. The native workbench invokes its shipped fixed worker families and local identity machinery. Neither path is silently substituted or initialized here. The preserved research scheduler is another distinct implementation. A local workbench extension is useful executable integration within its declared scope, not proof that an authenticated native service is operating.

6.2 Service and improvement have separate accounts

The custody layer maintains separately enforced service and improvement allowances. Earned capacity starts at zero. Reservations name one account; failed research cannot borrow the service balance. Measured CPU is settled in integer microseconds. Work beyond a reservation is recorded at its actual cost, refused and placed on accounting HOLD. The accounting does not physically preempt an indivisible native numerical call. Input-size/call bounds and observed overruns must therefore be reported honestly.

The frozen allowances per stream are 10,000 ms of service-dispatch CPU, 1,000 ms of external improvement-dispatch CPU and a 500 ms cumulative transfer cap. Each accepted B mission may release 50 ms; the eight B missions can therefore release at most 400 ms. Construction reserves 400 ms and restoration 600 ms. A separate 2,000 ms external control allowance covers shared startup, custody settlement and aggregate artifacts. The wall deadline is 300 s and the absolute local authority expiry 600 s. Dispatch, control and total-process measures retain their different boundaries; overlapping CPU measurements are not added twice.

Only a completed, accepted service request can release its unused reserved allowance into the earned account, subject to a cumulative cap. That transfer conserves the original endowment. It creates no money and does not itself prove that the capability saved compute: an overly generous reservation would also leave slack. The scientific resource certificate must separately identify the matched alternative, measured incremental benefit, available capacity and the subsequent construction that consumed it. Released external allowance and improvement-earned capacity are distinct labels unless that evidence supports their equivalence.

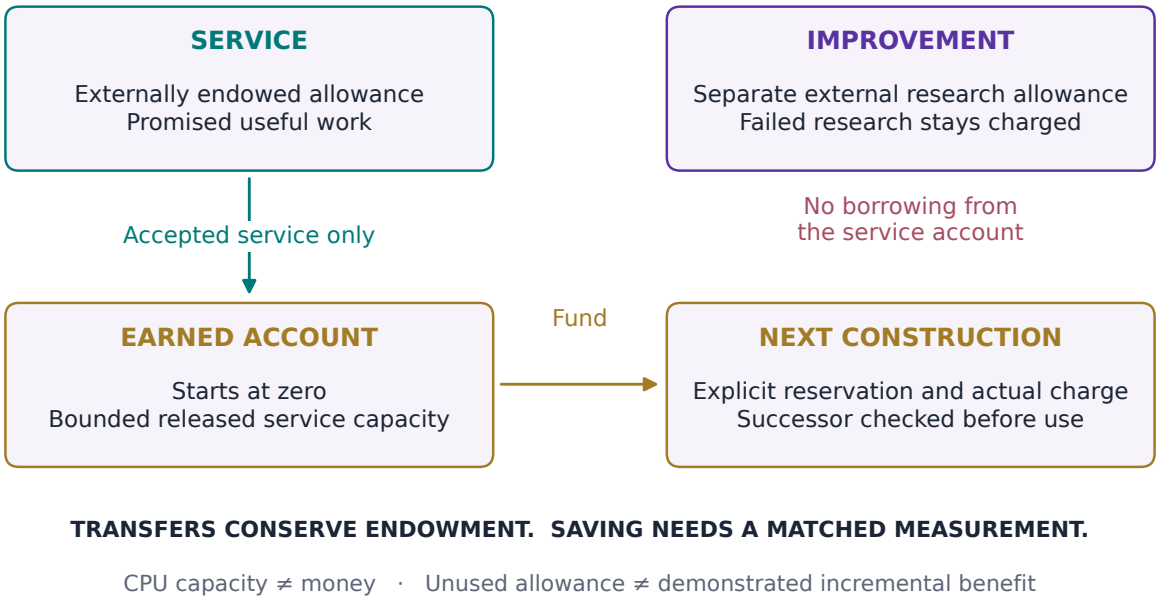


Figure 7: Three accounts, two origins. Research has its own external allowance. Earned capacity begins empty and can receive a bounded transfer from accepted service. The transfer is conserved resource custody; evidence of incremental saving is a separate causal requirement. No CPU quantity is converted into cash.

6.3 Recovery preserves cost and refusal

A single-writer local journal retains requests, receipts, account changes, stops and asset dispositions in a contiguous hash chain with a separately appended head anchor. A completed duplicate returns the same receipt without another execution instruction or charge. A pending reservation found after an interrupted process is quarantined and conservatively charged its reserved ceiling; its true lost CPU remains unknown. The system does not retry uncertain work under a fresh name and omit its cost.

Source or dependency changes, invalid receipts, stale checkpoints and exhausted allowances fail closed. STOP persists across restart. Explicit resume retains the stop history and the original absolute expiry; it cannot extend authority. A result arriving after STOP or expiry is retained for accounting but not accepted. Rollback selects a still-applicable earlier asset without rewinding history. Rejected descendants cannot re-enter through restored state, and a rejected parent invalidates dependent selection.

An actual process boundary. The frozen recovery probe uses the declared development network of 576 nodes, outside the efficacy confirmation. Worker PID 7 dispatches and independently checks A, persists its portable factor and accepted receipt, closes its Node bridge and exits with code 23 before acknowledgement. Worker PID 19 opens the same custody directory: repeating A returns the original receipt with no second execution; a fresh B load uses the restored factor through another workbench dispatch and passes the edge-based check. The retained execution ledger contains exactly one A and one B. All ten probe checks pass, including unchanged parent identity and empty inherited current proof and authority. Total worker-and-descendant CPU is 692.213 ms; wall time is 688.143 ms. This intentional post-commit exit tests lost acknowledgement across OS processes, not every possible crash window. Pending-commit quarantine is tested separately.

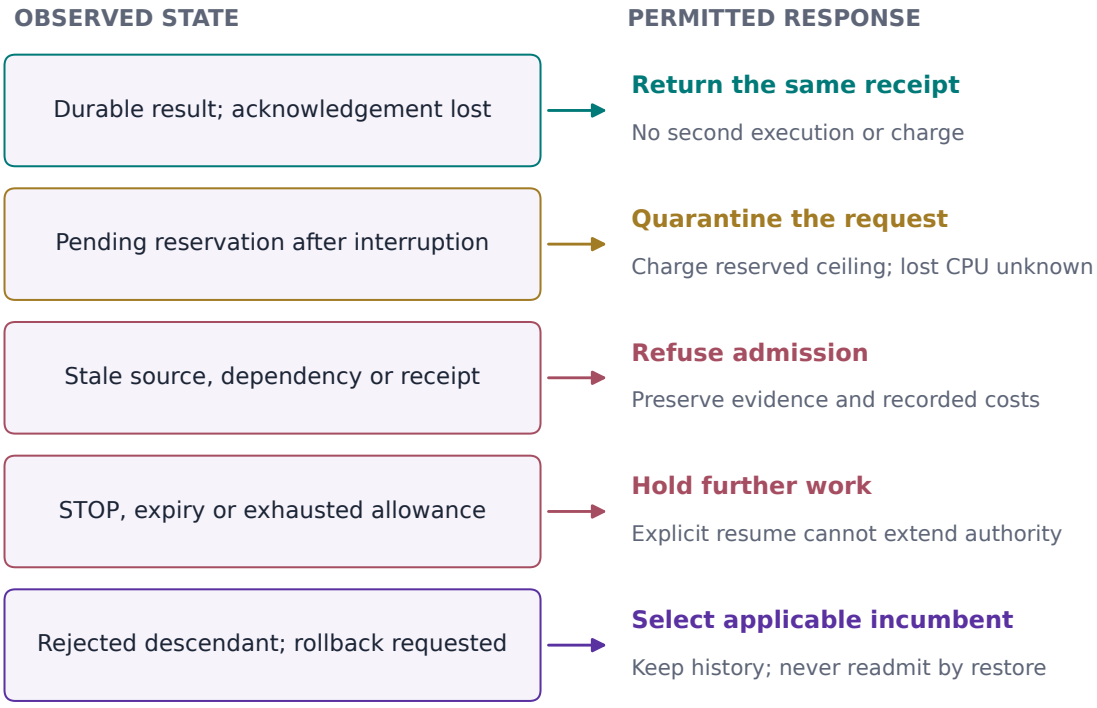


Figure 8: Recovery preserves the distinction between completed, uncertain and forbidden work. A lost acknowledgement after durable commitment returns the original result; an unresolved reservation is conservatively charged and quarantined. Rejection, STOP and expiry cannot be cleared by replaying history. The map describes tested local controls, not an external exactly-once service guarantee.

These are local integrity controls. The unsigned journal and anchor share one host; replacing both with an older matching pair cannot be detected without a separately retained checkpoint. No claim of cryptographic authentication or independence from the controlling OS account is made. The source-bound development fault records, frozen operational probe and scientific run's restoration observations retain their different scopes. Their complete records are in `runtime/development_bridge_tests.json`, `runtime/development_custody_tests.json`, `operations/process-recovery/PROCESS_RECOVERY.json` and `browser/qa/`.

6.4 A clear browser journey

The browser companion provides one obvious start: compute a bounded network response, reveal the check and cost, retain its method, change the load, construct a scoped successor and inspect fresh work. Its small local JavaScript calculation uses an 8×8 network and is separate from the native SciPy confirmation. Recorded study evidence is explicitly labelled replay. The existing Alpha `mission-store.mjs` supplies IndexedDB compare-and-swap history; saved state supports an explicit later resume. On reload, accepted outputs, assets and packs are rechecked. The unchanged upstream store holds at most twelve records; reaching its limit refuses a new run rather than silently discarding history. Closing the tab stops its computation.

The browser directly computes four fresh phases and permits the factor-caching direct reference to win. Its operation tariff includes numerical arithmetic, factor retrieval and edge checking, with hashing, serialization, rendering and persistence explicitly outside that tariff. Device times are separate. It enforces separate

Evidence set	Outcome	Scope and relevant boundary
Workbench bridge	10/10	Actual callback order; duplicate refusal; invalid source/receipt; callback failure; STOP before and after work.
Custody components	16/16	Journal, dependencies, distinct budgets, persistent STOP, expiry, rollback and pending quarantine. One expiry-during-work check injects a development clock.
Two OS processes	10/10	Durable A; deliberately lost acknowledgement; no duplicate A; portable parent used on fresh checked B.
Confirmation restoration	42/42	Six maintenance streams across seven arms; restored or rechecked methods produce qualified outputs.
Confirmation scope	168/168	One unsupported conductance case per stream and arm is refused; no unsupported solution enters qualified coverage.
Browser checker	17/17	Independent edge calculation and malformed input/output, changed dependency and ancestry checks.
Browser journey	21/21	Actual local computation, retained reference, downloads, mobile layout, STOP, resume and no outbound request.
Browser interruption	8/8	Real page reload with a pending reservation, conservative charge, durable-boundary resume and corrupted-state refusal.

Table 6: Separate operational evidence sets. These counts are checks or stream events, not independent efficacy samples. Component and browser records are development evidence; the OS probe executes the final frozen source on development inputs.

service and improvement ceilings but makes no earned-credit transfer claim. These operation counts are not equivalent to native CPU, money or the full cost of research.

Approved-executor mode remains disconnected. Local computation needs no account, API key or wallet. A saved receipt is not a running production service.

6.5 Historical evidence is not current authority

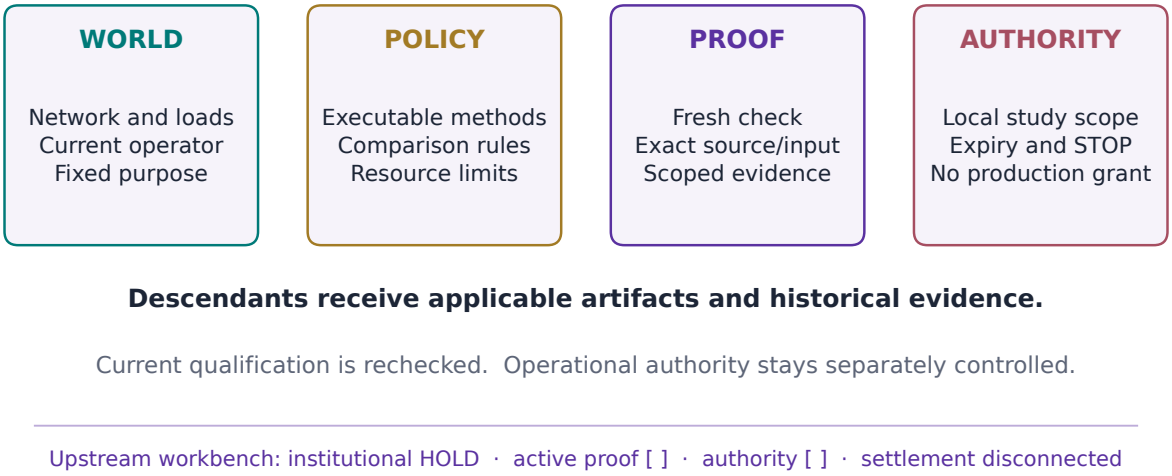


Figure 9: The four inspectable objects remain separate. A successor can inherit an applicable method and historical evidence. Its current output still needs a fresh check; institutional admission and operational authority are not inherited.

WORLD contains the supplied network and requirements. POLICY contains the fixed solver/construction policy and resource rules. PROOF contains exact numerical checks, comparison records and their independence limits. AUTHORITY contains only the bounded local study scope. The unchanged workbench pack records institutional `HOLD`, empty active authority and disconnected settlement. A study-level product decision does not edit that institutional status. An integrity hash identifies bytes; it is not an independent truth oracle.

6.6 Jobs, Manager, Fleet and applicable token utility

The actual inspected four-repository interfaces and five Alpha contract source identities are retained in the audit manifests [22, 23]. The workbench's existing Jobs compiler emits unfunded bounded drafts. Jobs' inspected receiver uses AGIALPHA18; Manager's distinct receiver uses USDC6. Fleet's control interface remains subject to its original source/review/action restrictions. None is invoked as a numerical executor or settlement service by this experiment.

Inspected contract	Relevant function and scope
AscensionAccess	`register`, `owns`, `setAdmission`, `eligible`: roles and expiring admission; no numerical proof.
NovaSeed	`seal`, `plan`, `creatorOf`: commitments and parent identity; no inherited current authority.
AscensionRiskOracle	`lockForJob`, `openReview`, `attest`, `green`: staked review/capacity; not an independent mathematical checker.
AscensionJobMarket	`post`, `award`, `submit`, `validate`, `timeout`: bounded funded work and settlement if separately authorized.
AscensionMark	`open`, `buy`, `route`, `collect`, `close`: funding/routing and closure; token movement is not productive value.

Table 7: Source inspection only. No deployed address, chain state, local EVM execution, funded draft, signed transaction or live settlement is established. No contract call is needed to solve these missions.

Practical $AGIALPHA utility would require separately authorized funded work and the applicable contract conditions. It is not inferred from this experiment's branding, token scarcity or price. The new edition preserves previous URLs and awaits its own exact-source review and publication process. Draft preparation, review, merge, public release and served-file verification are reported separately; original Harness/Fleet holds and the existing publication owner's sequencing remain binding.

7 What this flywheel has earned

7.1 A stronger mechanism, with a narrower economic conclusion

The new experiment resolves a concrete weakness in Strategy Succession: the retained object now participates in constructing a distinct capability, and that capability is used on fresh, qualified work. C is causally downstream of the actual accepted B08 response. Removing the parent while permitting exact reconstruction changes the measured work required to obtain the same successor. An independently constructed equivalent is also allowed, and conventional reuse receives the same algorithm vocabulary, information and resources. These interventions distinguish productive inheritance from a decorative ancestry label.

The scientific advance is this complete, inspectable construction path inside the existing workbench, coupled to an honest admission decision. It is not a new sparse-solver theorem, an autonomous discovery of the Woodbury identity, or evidence that a general-purpose agent beats equally enabled conventional learning. The numerical methods are established; their combination, inherited state, decision context and full costs are what this experiment tests. No language model generates algorithms during confirmation.

The distinction matters because the strongest conventional controls already solve every registered request. The flywheel has useful reuse and construction improvement within its declared scope, but no measured capability expansion in qualified coverage. It does not pass the whole-system productivity gate. Accumulating 18 correct instruments cannot reverse that decision. A system that preserves the incumbent and reports HOLD is performing a necessary part of selective improvement.

Three claims that must remain separate

Benchmark qualification: a method satisfies the frozen mathematical and scope checks.
Product adoption: the complete method clears the required benefit against the best available alternative. This experiment does not clear that gate.
Investment recovery: cumulative attributable benefit repays discovery, failed variants, development, integration, maintenance and human effort. Unknown human/model costs and unestablished incremental benefit leave recovery unresolved.

7.2 The bottleneck moved outside the algebra

The development screen correctly identified shared structure: repeated loads and small positive operator changes admit economical reuse. It did not adequately establish how much of the complete accountable workflow that structure could save. Native factorization was already cheap at these problem sizes, while input binding, workbench execution, verification, custody and persistence remained substantial.

A post-confirmation headroom calculation makes the design error tangible. In the recurring regime, the native C action averages 66.53 ms against 1,642.07 ms for the entire native stream, about 4.05%. Even eliminating that entire action would not alone establish a 5% stream improvement; other savings would also be needed. This is a retrospective diagnosis, not a preregistered impossibility bound. It explains why a visibly better construction step was an insufficient basis for an optimistic whole-system forecast.

The local benefit is nevertheless measurable. Across the same eighteen scoped streams, the inherited algorithm/solve/persistence phase averages 1.879 ms versus 11.570 ms after exact parent reconstruction. The corresponding complete C actions average 57.447 and 69.434 ms, with inheritance lower in seventeen streams. The independently constructed equivalent averages 10.462 ms and 69.466 ms. These post-run attribution summaries support a local mechanism; they do not replace the registered whole-stream analysis or supply a multiplicity-corrected new efficacy claim. The validation-path asymmetry described in Section 5 further limits interpretation of the native-constructor contrast.

7.3 Retention has a physical and operational price

The flywheel and conventional-reuse arms each retain 49,545,144 bytes in 48 unique physical capability files across 24 streams: about 2.06 MB per stream. Native refactorization retains 79,861,440 bytes in 48 files; exact rebuilding retains 88,767,528 bytes in 72; independently equivalent construction retains 72,626,520 bytes in 72. The direct specialist retains no factor NPZ files, although it still uses ordinary matrix and response state. These counts deduplicate logical aliases that refer to the same immutable path. They are capability storage, not the substantially larger raw research-evidence archive.

A smaller retained footprint is one useful property, not sufficient proof of value. The factor must remain loadable, applicable and bound to its dependencies. Maintenance pays for those obligations, and the matched maintenance regime exposes a case where direct CG is cheaper. The implementation bounds the number of missions and candidate assets; it does not demonstrate an indefinite archive with free retention or a long-horizon optimal retirement policy. Immutable historical evidence and a currently admissible executable are different objects. Neither automatically inherits fresh proof or operational authority.

7.4 What the feedback account actually proves

The resource ledger proves conservation and permitted reassignment: accepted service work can release unused externally reserved CPU allowance, and a subsequent bounded action can spend it without borrowing the separate research account. It does not prove that the method caused the released headroom. Controls receive the same generous reservations, and the aggregate point difference against native refactorization is only 280.073 ms across all streams, with uncertainty including no advantage. Calling the 9,600 ms reassigned allowance “earned savings” would overstate the evidence by both origin and amount.

Actual CPU accounting is closer to usable capacity than an invented operation currency, but one host’s unused reservation is still not cash, an operator’s recovered hour, or a market margin. There is no paid demand in this study. A prospective commercial decision would require measured arrival rates, service commitments, utilization, deployment costs and an actual price for the capacity or labor saved. Token supply and price are irrelevant to whether this numerical instrument creates productive value.

The complete research investment is also larger than the participant totals. The preserved record includes screening, six numerical variants, three bounded campaign attempts, two incomplete development failures, independent verification and publication work. The final raw audit alone adds 98.522 s of CPU. Human task definition, source inspection, debugging, review and setup occurred before or outside the automated run; their durations and model charges were not reliably measured. They are unknown costs, not zero costs.

7.5 Limits and the next falsifiable hypothesis

These are synthetic grounded networks on one host, with 24 independent seed streams and four prescribed regimes. Loads within a stream share an operator and are not independent replications. The tested finite

foreground loop does not establish unattended production uptime, distributed scalability or persistent operation after its process exits. The source-bound checker and second aggregator improve inspectability, but the commissioned team and host are shared. Instrumented parent calls are not an external CPU trace. The process-recovery probe and conservative pending-action quarantine establish specific failure behavior, not a general durable distributed transaction protocol.

The next hypothesis is specific and remains unexecuted: *when sparse factorization occupies a material share of whole-workflow cost, a retained response instrument can make small operator changes cheaper even after matched verification, persistence and maintenance.* A bounded three-dimensional network family is one candidate because its fill-in can make factorization more costly. The first experiment should be a small end-to-end headroom screen with the same cheap competent builder, native reuse and conventional learning. Reject the family if the maximum plausible removable cost cannot clear the required margin. Compact incremental provenance is another candidate improvement, but it must preserve the same guarantees and be available to all relevant comparators.

Any such test needs a new identity, new development cap, newly frozen analysis and untouched confirmation seeds. The present negative gate does not authorize another search on these 24 streams. Execution improvement and construction improvement should be established before a further claim that the discovery process itself improves. Increasing the rate of improvement would then require its own repeated, comparable measurements. A useful system need not accelerate forever; this one has not yet demonstrated acceleration.

Build a flywheel that earns its next turn

A mission can leave behind an executable means of doing later work. This experiment demonstrates that path and its verification. Earning the next turn requires the additional whole-system benefit that this campaign did not establish. The appropriate successor decision is to retain the evidence, keep competent service, and leave product admission on HOLD.

8 A usable companion and an auditable publication

8.1 One obvious start, three explicit execution states

The self-contained browser companion opens with a single **Start four missions** action. It requires no account, installation, wallet or API key. A small synthetic network is solved locally in JavaScript; the reader sees the verified response, retained factor, changed-network successor, fresh reuse, declared costs and a conventional comparison. The load can be changed. Evidence can be downloaded. STOP, interruption, restoration, stale-scope refusal and rollback are visible parts of the journey.

This fresh computation uses a 64-node browser workload and a declared operation tariff. It is a working explanatory implementation, not the native CPU experiment. The tariff excludes hashing, serialization, rendering and browser storage overhead. The separately labeled **Recorded confirmation** panel embeds the unmodified native analysis, exact source identities and the first registered A–B08–C–D01 lineage. It shows the failed product gate as prominently as the successful construction. Reading that panel does not execute the native campaign. An authenticated approved executor is explicitly disconnected. Closing the tab stops local execution; browser persistence does not imply a running service.

The browser's executable closure is frozen separately from presentation. Final recorded-result integration preserves that identity. Automated component, interface and actual reload checks are retained, together with a final 32-check presentation/download report. Desktop and mobile layouts were visually inspected. The interface neither claims a live hosted agent nor issues transactions.

8.2 Reproduce the evidence without changing its status

The complete companion contains source, protocols, original raw data, rejected development attempts, intervention records, the bibliography, editable SVG/PDF figures and this authoritative author PDF. Large raw records are compressed losslessly in an inner evidence archive; the supplied extraction command verifies its SHA-256 manifest before admitting files. The paper-only source package is sufficient to rebuild the publication and its figures without unpacking the numerical trial arrays.

The recorded environment is Python 3.12.14, NumPy 2.4.6, SciPy 1.17.0 and Node 24.22.1, with numerical thread counts fixed to one. Publication figures use Matplotlib 3.10.8; the executed paper build uses

XeLaTeX/BibTeX from TeX Live 2023 and Latin Modern fonts. Exact versions, platform observations and reproduction commands are in `protocol/ENVIRONMENT.json`, `requirements.txt` and `REPRODUCE.md`. Once dependencies are installed, the numerical campaign requires no network service.

First require the delivered `protocol/FREEZE.json` to exist, then verify its 32 file identities. The freeze script can also create a new freeze when none exists; that behavior must not be used to legitimize a modified or incomplete historical package. For read-only reconstruction of the result, run the producer aggregator and independently implemented raw auditor into a new output directory. Repeating the numerical missions uses `companion/confirm.py` with the original `SEEDS.json` supplied explicitly through `--seeds-file`. The runner labels this reproduction, not additional confirmation. CPU timings are new measurements and need not match historical observations byte for byte.

The source archive rebuild runs `paper/build.sh`, regenerates vector figures from packaged data, and executes XeLaTeX, BibTeX and reference-resolution passes. The delivery verification binds the canonical PDF to a clean packaged-source rebuild and a visual review of every final rendered page. PDF serving metadata can alter a displayed copy's bytes; the author PDF preserved inside the source archive is the authoritative identity for exact comparison.

8.3 Evidence docket and publication boundary

Claim or obligation	Primary inspectable artifact
Prior failure reconciled	`audit/PRIOR_FAILURE_AUDIT.json` and unchanged prior freeze
Prospective design and source	`protocol/CONFIRMATORY_PROTOCOL.json`, `protocol/FREEZE.json`
Single fresh campaign	`results/confirmation-v1/SEEDS.json` and the same directory's `EXECUTION.json`
Primary and comparator outcomes	`results/confirmation-v1/ANALYSIS.json` and complete trial records
Independent raw verification	`review/results/INDEPENDENT_RESULTS.json`
Construction attribution	`review/results/MECHANISM_ATTRIBUTION.json`
Quota, receipt and recovery claims	Trial custody journals; `operations/process-recovery/`
Failures and interventions	`development/ATTEMPT_LEDGER.json`, `protocol/INTERVENTIONS.json`
Browser state and source binding	`browser/BROWSER_EXECUTION_LOCK.json` and final QA records
Complete claim-to-artifact map	`EVIDENCE_DOCKET.json`; final payload and archive manifests

Table 8: The narrative is an index into evidence. Complete paths, identities, qualification limits and review statuses are supplied in the docket; an agent's favorable summary is not its own independent final proof.

The freeze identity is `0c9c3ad04ab99a992e0746c2416eb657a46ddfae99176e26d7d7245ea0e2280d`; its prospective coordination notice was posted in issue 4850 before seed generation[20]. The exact independent-auditor report identity begins `466e92056bf83f42`. The complete hashes and verified archive identities accompany the delivery. Intermediate milestones were preserved before final publication work, and durable copies are checked against their intended content identities.

This edition is an additive handoff under `studies/agi-alpha-flywheel-earned-turn-v1/`. It preserves the preferred Edition 02[24], the earlier Strategy Succession study and their historical URLs. Prepared source, owner review, merge, asset release and a verified served website are distinct states. At manuscript completion, the new edition is prepared for the existing review process; this paper does not claim owner approval, merge, public website deployment or removal of Harness/Fleet restrictions. The publication handoff records the subsequently observed repository state without retroactively granting execution authority.

BUILD WHAT MAKES THE NEXT MISSION BETTER.

Keep the result useful. Make the inheritance executable. Let the evidence decide the next turn.

References

[1] Vincent Boucher. AGI ALPHA FLYWHEEL: Strategy as a productive asset. Research and Engineering edition 03; additive source draft PR 4896, 2026. URL https://github.com/MontrealAI/AGI-Alpha-Agent-v0/pull/4896. 22-page canonical author PDF SHA-256 344b312678629076debae48be1573b3149e24c009144e521f45f8f777217cb73. Original frozen confirmation preserved; not reopened as new development.

[2] Vincent Boucher. AGI ALPHA: A Scalable Substrate for Intelligence Organizations. Canonical unified publication PDF, 2026. URL https://raw.githubusercontent.com/MontrealAI/agialpha-first-real-loop/main/docs/manuscript/AGI_ALPHA_Unified_Publication_Final.pdf. 198 PDF pages. Exact served PDF SHA-256 4b290d5a8232364b8808c0af8a96e7c9b152afd7ac3f5ba05668a538e073791a. Accessed 24 October 2026; source-reported results, not an independent replay.

[3] Vincent Boucher. Transformational Intelligence: Learning to Change the Game. October 2026 research manuscript, 2026. URL https://montrealai.github.io/AGI-Alpha-Agent-v0/Transformational_Intelligence.pdf. 24 PDF pages. Exact served PDF SHA-256 d7170e8217e3bf6aa1b84868c1c76c5994c6995c5297ed092d7e72a6430c664f. Accessed 24 October 2026; source-reported results, not an independent replay.

[4] MONTREAL.AI. The Possibility Foundry: Autonomous Enterprises That Invent Their Next Capability. Research edition 01, 2026. URL https://montrealai.github.io/AGI-Alpha-Agent-v0/papers/possibility-foundry-v1.pdf. 21 PDF pages. Exact served PDF SHA-256 906b63fe49294fc6c7fb4963f8cfea0e59863058c2f2845778e1bf8ad777ccca. Accessed 24 October 2026; source-reported results, not an independent replay.

[5] Vincent Boucher. Mission-Sovereign Successors: GoalOS Singularity Navigator Omega + SEIZE—Alpha, Beta, Spread, and the Verified Succession Institution in Singularity Time. Version 5.0.0, Authoritative Publication Edition, 2026. URL https://montrealai.github.io/goalos-singularity-navigator-omega-seize-agi-alpha-or-agi-club-owner-access-v1/research/GoalOS_Singularity_Navigator_Omega_SEIZE_Mission_Sovereign_Successors_v5_0_0_A4.pdf. 71 PDF pages. Exact served PDF SHA-256 9dd9caa6170bd1905e52926516d687c47313d58cd5e2a59e6038eabf0014639b. Accessed 24 October 2026; source-reported results, not an independent replay.

[6] MONTREAL.AI. Neural-Symbolic SUCCESSOR Omega: Customer-Owned Mission Intelligence. Requested URL edition with filename v0; document identifies itself as Definitive Corporate White Paper, 25 August 2026. No semantic version inferred from filename., 2026. URL https://montrealai.github.io/neuralsymbolicsuccessoromegav0.pdf. 59 PDF pages. Exact served PDF SHA-256 b1c0d9875bdbdb245ac71c39c80c97c8196370333f6856396d19a20b107699e9. Accessed 24 October 2026; source-reported results, not an independent replay.

[7] Vincent Boucher. Gym, Specialist ASI and Successor Omega: The Sovereign Mission Environment, the Proven Intelligence Institution, and the Successor to the Successor in Singularity Time. Version 2.1.0, US LETTER, 2026. URL https://montrealai.github.io/GoalOS_Gym_Specialist_ASI_Successor_Omega_v2_1_0_US_LETTER.pdf. 83 PDF pages. Exact served PDF SHA-256 98eabf107d9cde42e9f30b5ea2a62fdd4a2c461cd4ee1d755ec820492eb75ff2. Accessed 24 October 2026; source-reported results, not an independent replay.

[8] Vincent Boucher. SUCCESSOR Omega x AGI JOBS: The Constitutional Compiler and Proof-Carrying Work System for Mission-Sovereign Intelligence Institutions. Version 2.0.0; Dyadic Constitution, Job-Graph Algebra and Synthetic-Evidence Edition, 2026. URL https://montrealai.github.io/successor-omega-x-agi-jobs-v2.pdf. 110 PDF pages. Exact served PDF SHA-256 c25b911f20ac623bdb26f520376f2b1f0e774eba982db1c103a5e73d4b1d1a0d. Accessed 24 October 2026; source-reported results, not an independent replay.

[9] Vincent Boucher. Out-Learn: Change What You Need to Know. Revised research manuscript, 21 October 2026, 2026. URL https://montrealai.github.io/AGI-Alpha-Agent-v0/Out_Learn.pdf. 22 PDF pages. Exact served PDF SHA-256 01a8ea668f4c75cd0b5fafe3c5720e445bab7986abf27baee5bbba7d5e77febb. Accessed 24 October 2026; source-reported results, not an independent replay.

[10] Vincent Boucher. Out-Think: Transformational Intelligence for Inventing the Space of Reasoning. Research manuscript, 21 October 2026, 2026. URL https://montrealai.github.io/AGI-Alpha-Agent-v0/Out_Think.pdf. 22 PDF pages. Exact served PDF SHA-256 22280da9d3c74b995b0770b3b1a633924a0849d4f72fad51facd8f78c0336029. Accessed 24 October 2026; source-reported results, not an independent replay.

[11] Vincent Boucher. Out-Design: Creating the Freedom to Transform. Revised research manuscript, 21 October 2026, 2026. URL https://montrealai.github.io/AGI-Alpha-Agent-v0/Out_Design.pdf. 21 PDF pages. Exact served PDF SHA-256 65849e2f872b2dcdbc4a57baabb63e6a2644208c68dcc7c23ec0ee11ac1fe5dd. Accessed 24 October 2026; source-reported results, not an independent replay.

[12] Vincent Boucher. Out-Strategise: Transformational Intelligence for Constructing the Conditions of Success. Research manuscript, 20 October 2026, 2026. URL https://montrealai.github.io/AGI-Alpha-Agent-v0/Out_Strategise.pdf. 22 PDF pages. Exact served PDF SHA-256 8a848150f14c578106633324edb54bc59713a579c1318d9ea872d3d5b3ec7e77. Accessed 24 October 2026; source-reported results, not an independent replay.

[13] Vincent Boucher. Out-Execute: Constructing the Conditions of Completion. Revised research manuscript, 21 October 2026, 2026. URL https://montrealai.github.io/AGI-Alpha-Agent-v0/Out_Execute.pdf. 21 PDF pages. Exact served PDF SHA-256 85c6834e2dd496748f72a9099d3a70cb13047b49e09e3ed84ce911e1f1c530f3. Accessed 24 October 2026; source-reported results, not an independent replay.

[14] Vincent Boucher. Out-Evolve: Building the Means of Discovery and Production. Revised research manuscript, 20 October 2026, 2026. URL https://montrealai.github.io/AGI-Alpha-Agent-v0/Out_Evolve.pdf. 29 PDF pages. Exact served PDF SHA-256 5979bcaf704934ba42e4722389cbb1a9ef0cbc889e1b286428615a82a31e99bc. Accessed 24 October 2026; source-reported results, not an independent replay.

[15] SciPy developers. scipy.sparse.linalg.splu. SciPy 1.17.0 official reference documentation, 2026. URL https://docs.scipy.org/doc/scipy-1.17.0/reference/generated/scipy.sparse.linalg.splu.html. Version matched to the executed numerical environment. Accessed 24 October 2026.

[16] SciPy developers. scipy.sparse.linalg.cg. SciPy 1.17.0 official reference documentation, 2026. URL https://docs.scipy.org/doc/scipy-1.17.0/reference/generated/scipy.sparse.linalg.cg.html. Conjugate gradient interface, convergence criterion and preconditioner scope. Accessed 24 October 2026.

[17] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, et al. SciPy 1.0: Fundamental algorithms for scientific computing in python. *Nature Methods*, 17:261–272, 2020. doi: 10.1038/s41592-019-0686-2. URL https://doi.org/10.1038/s41592-019-0686-2.

[18] SciPy developers. scipy.sparse.linalg.spsolve_triangular. SciPy 1.17.0 official reference documentation, 2026. URL https://docs.scipy.org/doc/scipy-1.17.0/reference/generated/scipy.sparse.linalg.spsolve_triangular.html. Portable sparse triangular solves. Accessed 24 October 2026.

[19] William W. Hager. Updating the inverse of a matrix. *SIAM Review*, 31(2):221–239, 1989. doi: 10.1137/1031049. URL https://epubs.siam.org/doi/10.1137/1031049.

[20] MONTREAL.AI. Earn the next turn: Prospective source and protocol freeze, issue 4850, 2026. URL https://github.com/MontrealAI/AGI-Alpha-Agent-v0/issues/4850#issuecomment-6391784019. Posted before fresh seed generation; freeze 0c9c3ad04ab99a992e0746c2416eb657a46ddfae99176e26d7d7245ea0e2280d.

[21] MONTREAL.AI. AGI ALPHA v1.42.0: Successor workbench extension interface, 2026. URL https://github.com/MontrealAI/AGI-Alpha-Agent-v0/blob/c352bad0a0941fdccd490a7224e3efd1b8de45d5/docs/agent/SUCCESSOR_WORK.md. Exact unchanged execution closure retained in the companion runtime manifest. Local source execution is distinct from native installation.

[22] MONTREAL.AI. Alpha ascension contract source interfaces, 2026. URL https://github.com/MontrealAI/AGI-Alpha-Agent-v0/tree/c352bad0a0941fdccd490a7224e3efd1b8de45d5/contracts/ascension. Five source interfaces inspected; current-main identity matched. No deployed chain state or settlement verified.

[23] MONTREAL.AI contributors. Successor operating and publication coordination, issue 4850, 2026. URL https://github.com/MontrealAI/AGI-Alpha-Agent-v0/issues/4850. Source-specific owner restrictions and publication sequencing retained in the current runtime audit.

[24] Vincent Boucher. AGI ALPHA FLYWHEEL: From completed work to compounding capability. Preferred published Edition 02, 2026. URL https://montrealai.github.io/AGI-Alpha-Agent-v0/papers/agi-alpha-flywheel-acceleration-v2.pdf. Historical URL and served bytes preserved. New editions are additive.